



Универзитет „Св. Кирил и Методиј“ во Скопје
**ФАКУЛТЕТ ЗА ИНФОРМАТИЧКИ НАУКИ И
КОМПЈУТЕРСКО ИНЖЕНЕРСТВО**

Семинарска Работа
по предметот
Неструктурирани бази на податоците

Споредбена анализа на перформансите помеѓу PostgreSQL и MongoDB врз податочното множество Yelp

Ментор
Проф. Д-р Слободан Калајџиски

Изработиле
Јован Давалиев индекс бр. 226035
Никола Петрушевски индекс бр. 226052
Андреј Бајалски индекс бр. 221147
Григориј Ваљаков индекс бр. 223232

Скопје, Септември 2026

СОДРЖИНА

1. ВОВЕД.....	3
1.1 Проблематика.....	3
1.2 Зошто споредба помеѓу PostgreSQL и MongoDB?.....	3
1.3 Преглед на податочно множество.....	4
1.4 Референца на користена литература.....	4
2. МЕТОДОЛОГИЈА.....	5
2.1 Предпроцесирање на податоци.....	5
2.2 Модел на податоци во PostgreSQL.....	7
2.3 Модел на податоци во MongoDB.....	8
3. ДОБИЕНИ РЕЗУЛТАТИ - ДЕТАЛНА АНАЛИЗА НА ПРАШАЛНИЦИ.....	10
3.1 Едноставни прашалници.....	10
3.1.1 Прашалник бр.1.....	10
3.1.2 Прашалник бр.2.....	11
3.1.3 Прашалник бр.3.....	12
3.2 Сложени прашалници.....	13
3.2.1 Прашалник бр.4.....	13
3.2.2 Прашалник бр.5.....	15
3.2.3 Прашалник бр.6.....	18
3.2.4 Прашалник бр.7.....	21
3.3 Многу сложени прашалници.....	23
3.3.1 Прашалник бр.8.....	23
3.3.2 Прашалник бр.9.....	26
3.3.3 Прашалник бр.10.....	27
3.4 Споредба што ги вклучува сите прашалници.....	29
4. ДИСКУСИЈА.....	31
5. ЗАКЛУЧОК.....	31
6. КОРИСТЕНА ЛИТЕРАТУРА.....	32

1. ВОВЕД

1.1 Проблематика

Во современите информациски системи се создаваат и обработуваат големи количества на податоци, поради што изборот на соодветен систем за управување со бази на податоци претставува значаен фактор за перформансите на апликациите. Различните типови на бази на податоци користат различни модели за организација и пристап до податоците, што може да влијае на времето потребно за извршување на одредени операции и прашалници.

Проблематиката на овој проект произлегува од потребата да се утврди како изборот на различен тип база на податоци влијае врз перформансите при извршување на различни типови прашалници над исто податочно множество. За таа цел, во проектот се користи Yelp податочното множество, кое содржи различни типови на податоци и меѓусебно поврзани информации за корисници, бизниси, оценки, рецензии и други активности. Преку имплементација на истите или функционално еквивалентни прашалници во PostgreSQL [1] и MongoDB [2], се овозможува нивна директна споредба и анализа на времето потребно за нивно извршување.

1.2 Зошто споредба помеѓу PostgreSQL и MongoDB?

PostgreSQL и MongoDB претставуваат два различни пристапи кон складирање и обработка на податоци. PostgreSQL е релационен систем за управување со бази на податоци, кој користи табеларен модел и релации помеѓу ентитетите, додека MongoDB е документ-ориентирана NoSQL база на податоци, во која податоците се чуваат во документи со флексибилна структура.

Изборот на овие две технологии за споредба овозможува да се анализира влијанието на различните модели на податоци врз перформансите при извршување на различни типови прашалници. Особено е интересно да се испита нивното однесување при едноставни филтрирања, прашалници кои користат податоци од повеќе ентитети, како и посложени прашалници со групирање, сортирање и агрегирање на податоци.

Yelp податочното множество е погодно за ваква споредба бидејќи содржи голем број записи и разновидни меѓусебно поврзани податоци. Со имплементирање на податоците во двата системи и извршување на еквивалентни прашалници, може да се измери и спореди нивното време на извршување. На тој начин, целта не е да се прогласи една база за генерално подобра од другата, туку да се утврди како различните типови операции и структурата на податоците влијаат врз перформансите на PostgreSQL и MongoDB.

1.3 Преглед на податочно множество

Во овој проект се користи Yelp податочното множество [3], достапно преку Kaggle, кое содржи податоци за бизниси, корисници, рецензии, совети (tips) и пријавувања (check-ins).

Податочното множество е избрано поради неговата големина, разновидноста на податоците и постоењето на меѓусебни релации помеѓу различните типови записи, што го прави соодветно за споредба на релационен и документ-ориентиран систем за управување со бази на податоци.

Пред внесувањето во PostgreSQL и MongoDB, податоците беа претходно обработени со цел да се отстранат невалидните записи, да се трансформираат одредени атрибути и да се обезбеди соодветна структура за понатамошна анализа. По процесот на претпроцесирање, податочното множество што се користи во експериментите содржи вкупно **150.346 бизниси, 1.000.000 корисници, 1.999.994 рецензии, 754.835 совети (tips) и 131.930 check-in записи.**

Податоците за бизнисите содржат информации како идентификатор, име, адреса, град, држава, географска локација, оцена, број на рецензии, работен статус, категории и дополнителни атрибути. Податоците за корисниците содржат информации за нивниот идентификатор, бројот на добиени комплименти, бројот на рецензии и други карактеристики поврзани со нивната активност. Рецензиите ги поврзуваат корисниците со бизнисите и содржат оцена, текст и датум на рецензијата. Дополнително, tips содржат кратки текстуални совети поврзани со бизнисите и корисниците, додека check-ins ги претставуваат временските податоци за посетите на одредени бизниси.

Вака организираното податочно множество овозможува креирање на различни типови прашалници, почнувајќи од едноставни филтрирања и пребарувања, па сè до посложени операции кои вклучуваат поврзување на повеќе ентитети, групирање, сортирање и агрегирање. Истите или функционално еквивалентни прашалници се извршуваат во PostgreSQL и MongoDB, со цел да се овозможи споредба на нивните перформанси.

1.4 Референца на користена литература

Споредбата на релациони и NoSQL бази на податоци претставува предмет на повеќе истражувања, особено во контекст на перформансите при извршување на различни типови прашалници. Постоечките истражувања покажуваат дека изборот на систем за управување со бази на податоци може значително да влијае врз времето на извршување, во зависност од структурата на податоците, сложеноста на прашалниците и начинот на кој системот ги обработува податоците.

Makris et al. [4] спроведуваат експериментална споредба на MongoDB и PostgreSQL со цел да ги анализираат нивните перформанси при обработка на просторно-временски податоци. Во истражувањето се користи реално податочно множество обезбедено од MarineTraffic, додека перформансите се оценуваат преку времето на одговор при извршување на различни прашалници кои симулираат реални сценарија. Авторите ги тестираат двата системи во кластерска околина и дополнително го анализираат влијанието на индексите врз времето на извршување. Резултатите покажуваат дека PostgreSQL со PostGIS постигнува подобри перформанси во речиси сите тестирани прашалници, со просечен забрзувачки фактор од приближно 2,1 во однос на MongoDB.

Истражувањето исто така покажува дека користењето индекси може значително да го намали времето на одговор, особено кај MongoDB.

Поново истражување на Urnikienė, Steponavičienė и Atanasov [5] се фокусира на споредба на перформансите за читање помеѓу PostgreSQL и MongoDB во сценарија поврзани со електронска трговија. Авторите користат идентично податочно множество во двата системи и анализираат повеќе типови прашалници, вклучувајќи филтрирачки и аналитички прашалници. Експериментите се извршуваат во контролирана околина, при што секој тип прашалник се извршува повеќепати за да се обезбеди посигурна споредба. Резултатите покажуваат дека PostgreSQL има значителна предност кај покомплексните аналитички операции, особено кај прашалници кои вклучуваат повеќе критериуми, поврзување и агрегирање на податоци. Од друга страна, MongoDB покажува предност во однос на искористувањето на простор за складирање, што укажува дека перформансите не зависат само од времето на извршување, туку и од карактеристиките на конкретниот систем и работното оптоварување.

Двете истражувања се особено релевантни за овој проект бидејќи користат сличен експериментален пристап: истите податоци се организираат во PostgreSQL и MongoDB, над нив се извршуваат еквивалентни прашалници и потоа се споредуваат добиените перформанси. Сличен пристап е применет и во овој проект, но со Yelp податочното множество и со различни категории на прашалници, поделени на едноставни, сложени и многу сложени прашалници. На тој начин, проектот се надоврзува на постоечките истражувања, но врши сопствена анализа на перформансите на PostgreSQL и MongoDB врз големо и разновидно податочно множество составено од бизниси, корисници, рецензии, tips и check-ins.

2. МЕТОДОЛОГИЈА

Сите скрипти, конфигурации и резултати од ова истражување се достапни на:

https://github.com/Hekaton003/NBP_Proekst

2.1 Предпроцесирање на податоци

Пред складирањето на податоците во PostgreSQL и MongoDB беше извршен процес на претпроцесирање со цел да се обезбеди конзистентност на податоците, да се обработат сложените структури и да се отстранат записите кои не ги задоволуваат условите за понатамошно користење. Бидејќи Yelp податочното множество е обезбедено во JSON формат, податоците беа прочитани и обработени со помош на Python [6] и библиотеката Pandas [7]. Поради големината на датотеките, кај User и Review податоците беше користен ограничен број записи при нивното вчитување. За User беа вчитани 1.000.000 записи, додека за Review беа вчитани 2.000.000 записи. Business, Tip и Check-in податоците беа вчитани како целосни датотеки.

Кај Business податоците беше извршена дополнителна обработка на сложените полиња **attributes** и **hours**. Бидејќи овие полиња содржат сложени речници, беше применета рекурзивна функција за нивно израмнување (flattening), при што сложените вредности се

трансформираат во одделни колони. На пример, сложените атрибути се претвораат во колони со соодветни имиња, додека работното време се организира во посебни колони за секој ден од неделата. Со ова се добива поедноставна и постројкурирана форма на податоците, погодна за понатамошно складирање и извршување на прашалници. По оваа трансформација, Business податочната рамка содржи 60 колони.

Дополнително, полето `categories`, кое во оригиналните податоци е претставено како текстуална вредност со категории одделени со запирки, беше трансформирано во листа од поединечни категории. Ова овозможува полесно пребарување и филтрирање на бизнисите според нивните категории. Кај Business податоците беа идентификувани недостасувачки вредности во полињата `attributes`, `categories` и `hours`. По обработката на овие полиња, преостанатите недостасувачки вредности беа заменети со вредноста `Unknown`, со што се избегнува губење на записи и се обезбедува унифицирана вредност за податоците што не се достапни.

Кај User податоците беше обработено полето `elite`, кое во оригиналната структура содржи листа на години во кои корисникот имал Elite статус. Наместо оваа информација да се задржи како една текстуална вредност, таа беше трансформирана во четири одделни полиња: `elite_years_count`, `elite_first_year`, `elite_last_year` и `is_elite`. На овој начин се добива бројот на години во кои корисникот бил Elite, првата и последната година на Elite статус, како и логичка вредност која покажува дали корисникот воопшто има Elite статус. За корисниците без Elite статус, бројот на години се поставува на 0, додека за првата и последната година се користи вредноста `Unknown`, а `is_elite` се поставува на `False`.

За сите пет типови податоци беше извршена проверка на недостасувачките вредности. Кај Check-in податоците не беа пронајдени недостасувачки вредности, додека кај User, Tip и Review податоците исто така не беа идентификувани missing values во нивните полиња.

Следниот чекор беше проверка на референцијалната поврзаност помеѓу податоците. Кај Review записите беа пронајдени 6 записи со `user_id` што не постои во User податочната рамка, додека сите референци кон Business беа валидни. Кај Tip записите беа пронајдени 154.080 записи со непостоечки `user_id`, додека сите референци кон Business беа валидни. Кај Check-in податоците не беше пронајдена ниту една референца кон непостоечки Business запис. За да се зачува конзистентноста на податоците и валидноста на врските помеѓу ентитетите, Review и Tip записите што се однесуваат на непостоечки корисници беа отстранети. По оваа операција, бројот на Review записи се намали од 2.000.000 на **1.999.994**, додека бројот на Tip записи се намали од 908.915 на **754.835**.

Кај Check-in податоците, полето `date`, кое првично содржи повеќе датуми запишани како една текстуална вредност, беше трансформирано во листа од одделни датуми. На овој начин се зачувува информацијата за повеќекратните пријавувања на истиот бизнис, а структурата е соодветна за понатамошно складирање во базите на податоци.

На крајот беше извршена проверка на дупликати. Не беа пронајдени дупликати кај User, Business и Review податоците. Кај Tip податоците беа идентификувани 59 дуплирани записи, кои беа отстранети со цел секој запис да се појавува само еднаш. По отстранувањето, проверката покажа дека повеќе нема дупликати во Tip податоците.

По завршувањето на претпроцесирањето, обработените податоци беа зачувани во одделни CSV датотеки: `users.csv`, `businesses.csv`, `reviews.csv`, `tips.csv` и `checkins.csv`. Овие датотеки претставуваат финална верзија на податоците која понатаму се користи за моделирање и внесување во PostgreSQL и MongoDB, како и за извршување и споредба на различните прашалници.

2.2 Модел на податоци во PostgreSQL

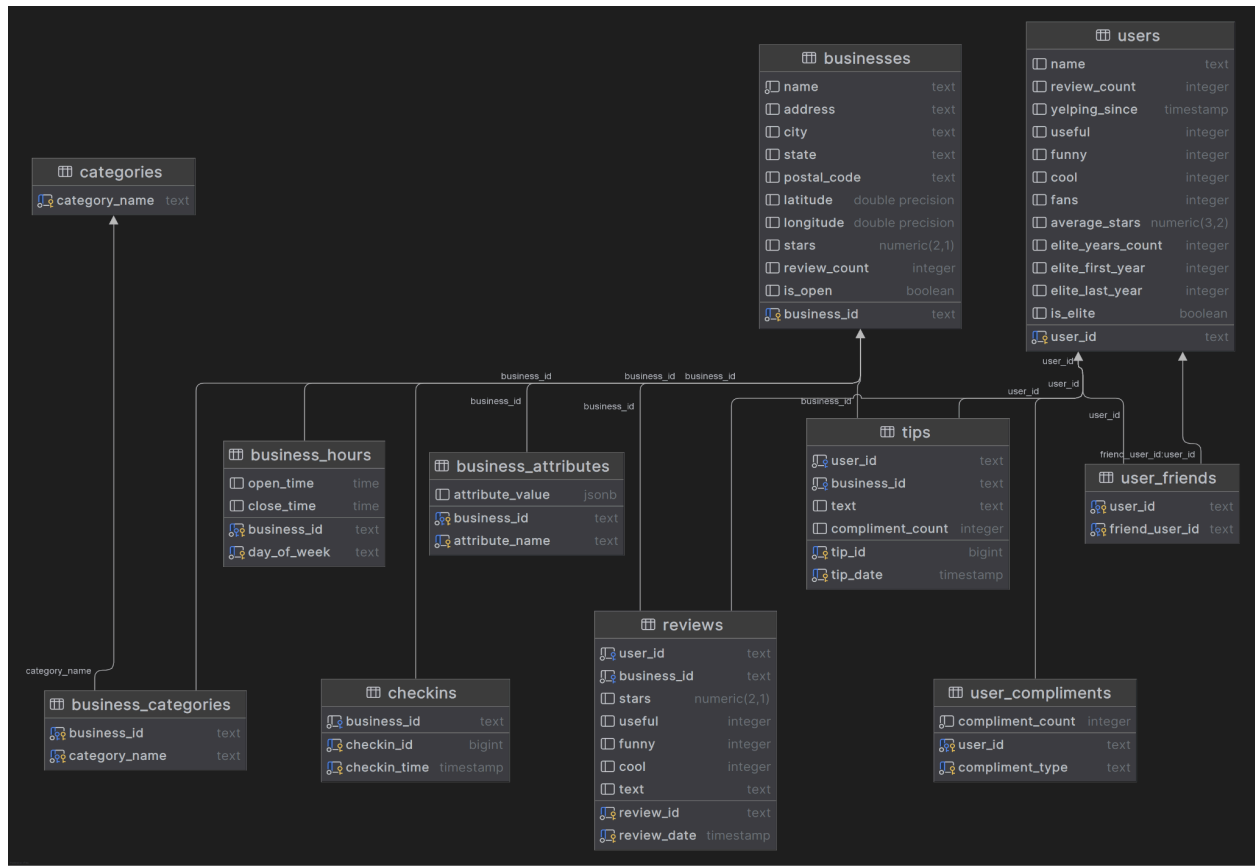
За складирање на Yelp податочното множество во PostgreSQL е креиран релационен модел составен од повеќе меѓусебно поврзани табели прикажано на Слика 1. Главните ентитети се претставени преку табелите `businesses`, `users`, `reviews`, `tips` и `checkins`. Дополнително, за нормализирање на податоците и одделување на посложените структури се користат табелите `business_categories`, `categories`, `business_attributes`, `business_hours`, `user_friends` и `user_compliments`.

Табелата `businesses` ги содржи основните информации за бизнисите и претставува централна табела за повеќе од поврзаните податоци. Со неа се поврзани категориите, атрибутите, работното време, рецензиите, `tips` и `check-ins` преку `business_id`. Од друга страна, табелата `users` ги содржи информациите за корисниците и е поврзана со `reviews`, `tips`, `user_friends` и `user_compliments` преку `user_id`.

За категориите е креирана посебна табела `categories`, додека врската помеѓу бизнисите и категориите е реализирана преку `business_categories`. На сличен начин, сложените атрибути на бизнисите се издвоени во `business_attributes`, а работното време е организирано во `business_hours`. Овој пристап овозможува податоците да бидат организирани во одделни релации и да се избегне непотребно повторување на истите информации.

Табелата `reviews` ги поврзува корисниците со бизнисите и содржи информации за оценката, корисноста, забавноста и кулноста на рецензијата, како и текстот и датумот на нејзиното објавување. Табелата `tips` на сличен начин содржи кратки коментари поврзани со корисници и бизниси. `Checkins` ги содржи информациите за пријавувањата на бизнисите, при што времето на секое пријавување е претставено како посебен запис.

Поради големиот обем на временски податоци, табелите `reviews`, `tips` и `checkins` се партиционирани по години со `PARTITION BY RANGE`, според колоните `review_date`, `tip_date` и `checkin_time`. Креирани се годишни партиции за однапред дефиниран временски опсег, како и `DEFAULT` партиција за записи што не припаѓаат во ниту една од нив. Податоците во `reviews` опфаќаат период од 2005 до 2022 година, додека податоците во `tips` и `checkins` се од 2009 до 2022 година. Овој пристап овозможува поефикасно извршување на пребарувања филтрирани според временски период и поедноставно управување со големите табели.



Слика бр.1 Entity-Relationship дијаграм за PostgreSQL база

2.3 Модел на податоци во MongoDB

Во предложената шема на база на податоци во MongoDB, прикажана на Слика 2, се идентификуваат четири основни колекции: **reviews**, **tips**, **businesses** и **users**. Податоците во колекциите се поврзуваат преку идентификаторите **business_id** и **user_id**, што овозможува поврзување на информациите за корисниците, бизнисите и нивните интеракции.

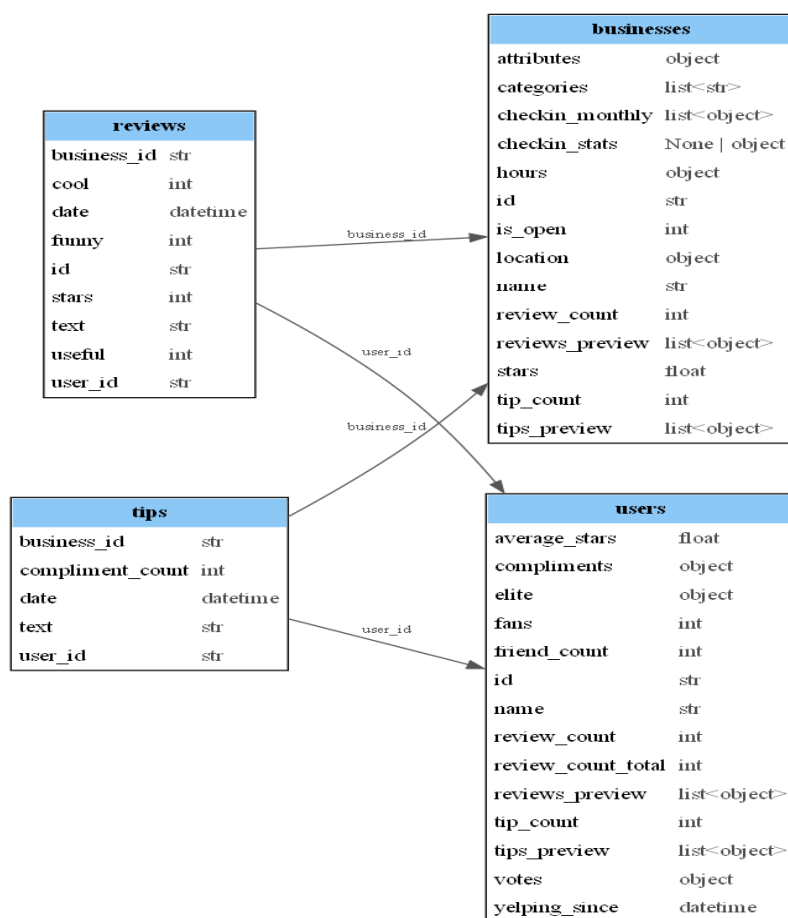
Колекцијата **reviews** ги содржи формалните кориснички повратни информации за бизнисите преку текстуални коментари, оценки во ѕвезди и метрики за корисност, хумор и „cool“ фактор. Преку **business_id** и **user_id**, рецензиите се поврзуваат со соодветните бизниси и корисници, што овозможува анализа на оценките и активноста на корисниците.

Колекцијата **tips** ги моделира кратките текстуални совети поврзани со корисниците и бизнисите. Покрај текстот, таа содржи информации за бројот на добиени комплименти и датумот на внесување. На овој начин, tips претставуваат дополнителен тип интеракција помеѓу корисниците и бизнисите, покрај формалните рецензии.

Колекцијата **businesses** претставува една од централните колекции во моделот и ги содржи основните информации за бизнисите, како што се локацијата, категориите,

работното време и дополнителните атрибути. Покрај основните податоци, колекцијата содржи и агрегирани статистики поврзани со активностите на корисниците, односно бројот на рецензии, tips и check-ins за секој бизнис. На овој начин, во рамките на еден документ се обединети најважните информации за бизнисот и неговата активност на платформата, што овозможува поедноставен пристап при извршување на прашалници.

Колекцијата **users** ги содржи информациите за корисниците и нивната активност на платформата. Покрај основните податоци и статистиките поврзани со рецензиите, како што се бројот на рецензии и просечната оценка, таа содржи и агрегирани информации за активността на корисниците преку tips. Дополнително, се чуваат податоци за комплиментите, пријателите, фановите и Elite статусот на корисниците. Со ваквата организација, информациите поврзани со активността на еден корисник се групирани во рамките на неговиот документ.



Слика бр.2 Документ-ориентиран модел на податоците во MongoDB

3. ДОБИЕНИ РЕЗУЛТАТИ - ДЕТАЛНА АНАЛИЗА НА ПРАШАЛНИЦИ

Во овој дел се презентирани резултатите од извршување на 10 прашалници категоризирани во три групи: едноставни, сложени и многу сложени користејќи ги PostgreSQL и MongoDB системите за менаџирање со бази на податоци.

3.1 Едноставни прашалници

3.1.1 Прашалник бр.1

Пронајдете бизниси во Филаделфија кои моментално се отворени и имаат повеќе од 100 рецензии.

PostgreSQL имплементација:

```
SELECT business_id, name, address, stars, review_count
FROM businesses
WHERE city = 'Philadelphia'
AND is_open = TRUE
AND review_count > 100
ORDER BY stars DESC, review_count DESC
```

MongoDB имплементација:

```
CITY = "Philadelphia"
philly_open_over_100_reviews = bench("philly_open_over_100_reviews",
  lambda: db.businesses.find(
    {"location.city": CITY,
     "is_open": 1,
     "review_count": {"$gt": 100}},
    {"_id": 0, "id": 1, "name": 1, "stars": 1, "review_count": 1}
  ).sort([("review_count", -1)]))
```

Споредба на перформанси:

Метрика	PostgreSQL	MongoDB
Број на резултати	1597	507
Минимално време (ms)	27.816	395.83
Максимално време (ms)	29.41	442.46
Медијана на времето на извршување (ms)	28.474	421.05

Број на извршувања	10	10
--------------------	----	----

PostgreSQL покажува значително подобри перформанси од MongoDB. Иако PostgreSQL враќа **1597 резултати**, а MongoDB **507**, времето на извршување кај PostgreSQL е многу пократко: медијаната изнесува **28.474 ms**, додека кај MongoDB е **421.05 ms**. И минималното и максималното време се значително пониски кај PostgreSQL (**27.816–29.41 ms**) во споредба со MongoDB (**395.83–442.46 ms**). Ова покажува дека за овој тип на едноставно филтрирање врз повеќе атрибути, **PostgreSQL е значително поефикасен**, со приближно **15 пати пократко медијанско време на извршување** од MongoDB, при што резултатите се добиени од 10 извршувања на секое барање.

3.1.2 Прашалник бр.2

Пронајдете рецензии со оценка од пет ѕвездички напишани во текот на 2018 година.

PostgreSQL имплементација:

```
SELECT review_id, user_id, business_id, stars, review_date, text
FROM reviews
WHERE stars = 5.0
      AND review_date >= TIMESTAMP '2018-01-01'
      AND review_date < TIMESTAMP '2019-01-01'
ORDER BY review_date DESC
```

MongoDB имплементација:

```
YEAR_START = datetime(2018, 1, 1, tzinfo=timezone.utc) # inclusive
YEAR_END   = datetime(2019, 1, 1, tzinfo=timezone.utc) # exclusive

five_star_reviews_2018 = bench("five_star_reviews_2018",
    lambda: db.reviews.find(
        {"stars": 5,
         "date": {"$gte": YEAR_START, "$lt": YEAR_END}},
        {"_id": 0, "id": 1, "business_id": 1, "user_id": 1, "stars": 1,
         "date": 1}
    )
)
```

Споредба на перформанси:

Метрика	PostgreSQL	MongoDB
Број на резултати	135090	135090

Минимално време (ms)	2071.588	4630.3
Максимално време (ms)	2330.83	5420.85
Медијана на времето на извршување (ms)	2106.877	5168.93
Број на извршувања	10	10

За прашањето „Пронајдете рецензии со оценка од пет ѕвездички напишани во текот на 2018 година“, PostgreSQL повторно покажува подобри перформанси од MongoDB, при што **двете бази враќаат ист број на резултати (135.090)**. Медијаната на времето на извршување кај PostgreSQL изнесува **2.106,877 ms**, додека кај MongoDB е **5.168,93 ms**, што значи дека PostgreSQL ја извршува оваа операција приближно **2,45 пати побрзо**. Дополнително, PostgreSQL има и пониско минимално време (**2.071,588 ms** наспроти **4.630,3 ms**) и пониско максимално време (**2.330,83 ms** наспроти **5.420,85 ms**). Резултатите од 10 извршувања покажуваат дека PostgreSQL е поефикасен и поконзистентен при пребарување на голем број рецензии според оценка и временски период.

3.1.3 Прашалник бр.3

Пронајдете деловни објекти во Филаделфија што располагаат со паркинг.

PostgreSQL имплементација:

```
SELECT b.business_id, b.name, b.address, b.stars, b.review_count
FROM businesses AS b
JOIN business_attributes AS ba
ON ba.business_id = b.business_id
WHERE b.city = 'Philadelphia'
AND ba.attribute_name = 'BusinessParking'
AND ba.attribute_value @> '{"lot": true}':::jsonb
ORDER BY b.stars DESC, b.review_count DESC
```

MongoDB имплементација:

```
CITY = "Philadelphia"
philly_with_parking_lot = bench("philly_with_parking_lot",
    lambda: db.businesses.find(
        {"location.city": CITY,
         "attributes.BusinessParking": {"$regex": r"'lot':\s*True"}},
        {"_id": 0, "id": 1, "name": 1, "attributes.BusinessParking": 1}
    ))
```

Споредба на перформанси:

Метрика	PostgreSQL	MongoDB
Број на резултати	1565	1565
Минимално време (ms)	106.076	369.34
Максимално време (ms)	121.19	652.9
Медијана на времето на извршување (ms)	110.875	401.24
Број на извршувања	10	10

За прашањето „Пронајдете деловни објекти во Филадельфија што располагаат со паркинг“, PostgreSQL повторно покажува подобри перформанси од MongoDB, при што **двете бази враќаат ист број на резултати (1.565)**. Медијаната на времето на извршување кај PostgreSQL изнесува **110,875 ms**, додека кај MongoDB е **401,24 ms**, што значи дека PostgreSQL ја извршува операцијата приближно **3,6 пати побрзо**. PostgreSQL има и пониско минимално време (**106,076 ms** наспроти **369,34 ms**) и максимално време (**121,19 ms** наспроти **652,9 ms**). Резултатите од 10 извршувања покажуваат дека PostgreSQL е не само побрз, туку и поконзистентен при филтрирање на деловни објекти според локација и достапност на паркинг.

3.2 Сложени прашалници

3.2.1 Прашалник бр.4

Ресторани во Филадельфија чии последни 50 рецензии имаат оценка од 4+ ѕвездички и го содржат зборот „бранч“ (brunch), со наведени имиња на рецензентите.

PostgreSQL имплементација:

```
SELECT
    r.review_id,
    r.review_date,
    r.stars AS review_stars,
    r.text,
    u.user_id,
    u.name AS reviewer_name,
    b.business_id,
    b.name AS restaurant_name,
    b.address
FROM reviews AS r
JOIN users AS u
```

```

ON u.user_id = r.user_id
JOIN businesses AS b
ON b.business_id = r.business_id
JOIN business_categories AS bc
ON bc.business_id = b.business_id
WHERE b.city = 'Philadelphia'
AND bc.category_name = 'Restaurants'
AND r.stars >= 4.0
AND r.text ILIKE '%brunch%'
ORDER BY r.review_date DESC
LIMIT 50

```

MongoDB имплементација:

```

CITY = "Philadelphia"
philly_brunch_mentions = bench("philly_brunch_mentions",
    lambda: db.businesses.aggregate([
        {"$match": {
            "location.city": CITY,
            "categories": "Restaurants",
            "reviews_preview": {"$elemMatch": {
                "text": {"$regex": "brunch", "$options": "i"},
                "stars": {"$gte": 4}}}},
        {"$project": {
            "_id": 0, "name": 1, "stars": 1, "review_count": 1,
            "mentions": {"$filter": {
                "input": "$reviews_preview", "as": "r",
                "cond": {"$and": [
                    {"$gte": ["$$r.stars", 4]},
                    {"$regexMatch": {"input": "$$r.text", "regex":
"brunch",
                                "options": "i"}}]}}}}},
        {"$addFields": {"n_mentions": {"$size": "$mentions"},
            "reviewers": "$mentions.user_name"}},
        {"$project": {"mentions": 0}},
        {"$sort": {"n_mentions": -1, "review_count": -1}},
        {"$limit": 25},
    ], allowDiskUse=True))

```

Споредба на перформанси:

Метрика	PostgreSQL	MongoDB
Број на резултати	50	25
Минимално време (ms)	2157.407	530.58
Максимално време (ms)	2292.932	650.58
Медијана на времето на извршување (ms)	2261.153	568.19
Број на извршувања	10	10

За прашањето „Ресторани во Филаделфија чии последни 50 рецензии имаат оценка од 4+ ѕвездички и го содржат зборот „бранч“ (brunch), со наведени имиња на рецензентите“, MongoDB покажува пократко време на извршување, но истовремено враќа помал број на резултати во споредба со PostgreSQL. PostgreSQL има медијана од **2.261,153 ms**, додека MongoDB има медијана од **568,19 ms**, што укажува дека MongoDB ја извршува операцијата приближно **4 пати побрзо**. Меѓутоа, PostgreSQL враќа **50 резултати**, додека MongoDB враќа само **25**, што е значајна разлика бидејќи целта на барањето е да се добијат **сите записи што ги исполнуваат условите**, а не само дел од нив. Оттука, иако MongoDB има подобро време на извршување, помалиот број на добиени резултати укажува дека резултатот не е еквивалентен со оној на PostgreSQL и може да значи дека PostgreSQL има предност во однос на целосноста на резултатите .

3.2.2 Прашалник бр.5

Пронајдете корисници кои оцениле најмалку 3 различни деловни објекти во истиот град, вклучувајќи ги информациите за корисникот и објектите што ги оцениле.

PostgreSQL имплементација:

```
WITH user_businesses AS (  
    SELECT DISTINCT  
        r.user_id, b.city, b.business_id, b.name AS  
business_name, b.address  
    FROM reviews AS r  
    JOIN businesses AS b  
        ON b.business_id = r.business_id  
    WHERE b.city IS NOT NULL  
) ,  
qualified_users AS (  
    SELECT user_id, city, COUNT(*) AS  
distinct_businesses_reviewed
```

```

        FROM user_businesses
        GROUP BY user_id, city
        HAVING COUNT(*) >= 3
    )
    SELECT
        u.user_id, u.name AS user_name, u.review_count AS
total_reviews_written,
        u.average_stars, q.city, q.distinct_businesses_reviewed,
        ub.business_id, ub.business_name, ub.address
    FROM qualified_users AS q
    JOIN users AS u
        ON u.user_id = q.user_id
    JOIN user_businesses AS ub
        ON ub.user_id = q.user_id
        AND ub.city = q.city
    ORDER BY q.distinct_businesses_reviewed DESC, u.user_id,
q.city, ub.business_name

```

MongoDB имплементација:

```

users_3plus_businesses_same_city =
bench("users_3plus_businesses_same_city",
    lambda: db.reviews.aggregate([
        {"$group": {"_id": "$business_id", "users": {"$addToSet":
"$user_id"}}},
        {"$lookup": {
            "from": "businesses",
            "localField": "_id", "foreignField": "id",
            "as": "biz",
            "pipeline": [{"$project": {"_id": 0, "name": 1,
"city": "$location.city"}}]}},
        {"$unwind": "$biz"},
        {"$unwind": "$users"},
        {"$group": {"_id": {"user_id": "$users", "city": "$biz.city"},
            "businesses": {"$addToSet": {"business_id": "$_id",
"name":
"$biz.name"}}}},
        {"$match": {"$expr": {"$gte": [{"$size": "$businesses"}, 3]}},
        {"$addFields": {"n_businesses": {"$size": "$businesses"}}},

```

```

        {"$sort": {"n_businesses": -1}},
        {"$limit": 100},
        {"$lookup": {
            "from": "users",
            "localField": "_id.user_id", "foreignField": "id",
            "as": "user",
            "pipeline": [{"$project": {"_id": 0, "id": 1, "name": 1,
"fans": 1,
"review_count": 1,
"average_stars": 1}}]}},
        {"$unwind": "$user"},
        {"$project": {"_id": 0, "city": "$_id.city", "user": 1,
            "n_businesses": 1, "businesses": 1}},
    ], allowDiskUse=True),
)

```

Споредба на перформанси:

Метрика	PostgreSQL	MongoDB
Број на резултати	690265	100
Минимално време (ms)	17718.352	89556.52
Максимално време (ms)	19289.407	95679.78
Медијана на времето на извршување (ms)	18002.621	91505.04
Број на извршувања	10	10

За ова прашање, PostgreSQL покажува значително подобри перформанси и во однос на времето на извршување и во однос на целосноста на резултатите. PostgreSQL има медијана на времето на извршување од **18.002,621 ms**, додека MongoDB има медијана од **91.505,04 ms**, што значи дека PostgreSQL ја извршува операцијата приближно **5,1 пати побрзо**. Дополнително, PostgreSQL враќа **690.265 резултати**, додека MongoDB враќа само **100 резултати**. Затоа, иако PostgreSQL има подолго апсолутно време на извршување од претходните едноставни барања, во овој случај тој покажува **значително подобри перформанси и многу повисока целосност на резултатите**, што го прави посоодветен за вакви комплексни операции со групирање, поврзување и обработка на голем број записи.

3.2.3 Прашалник бр.6

Картичка со профил на деловен објект — еден објект + последни 50 рецензии со имиња на рецензентите + последни 10 препораки (tips) + вкупен број на пријавувања (check-ins) + работно време + карактеристики.

PostgreSQL имплементација:

```
WITH selected_business AS (  
    SELECT *  
    FROM businesses  
    WHERE business_id = 'VQcCL9PiNL_wkGf-uF3fjg'  
),  
review_rows AS (  
    SELECT  
        r.review_id,  
        r.review_date,  
        r.stars,  
        r.text,  
        u.user_id,  
        u.name AS user_name  
    FROM reviews AS r  
    JOIN selected_business AS b  
        ON b.business_id = r.business_id  
    JOIN users AS u  
        ON u.user_id = r.user_id  
    ORDER BY r.review_date DESC  
    LIMIT 50  
),  
latest_reviews AS (  
    SELECT jsonb_agg(  
        jsonb_build_object(  
            'review_id', review_id,  
            'review_date', review_date,  
            'stars', stars,  
            'text', text,  
            'user_id', user_id,  
            'user_name', user_name  
        )  
    ORDER BY review_date DESC  
    ) AS data  
    FROM review_rows
```

```

),
tip_rows AS (
    SELECT
        t.tip_id,
        t.tip_date,
        t.text,
        t.compliment_count,
        u.user_id,
        u.name AS user_name
    FROM tips AS t
    JOIN selected_business AS b
        ON b.business_id = t.business_id
    JOIN users AS u
        ON u.user_id = t.user_id
    ORDER BY t.tip_date DESC
    LIMIT 10
),
latest_tips AS (
    SELECT jsonb_agg(
        jsonb_build_object(
            'tip_id', tip_id,
            'tip_date', tip_date,
            'text', text,
            'compliment_count', compliment_count,
            'user_id', user_id,
            'user_name', user_name
        )
    ) AS data
    ORDER BY tip_date DESC
    FROM tip_rows
),
checkin_summary AS (
    SELECT
        COUNT(*) AS total_checkins,
        MIN(c.checkin_time) AS first_checkin,
        MAX(c.checkin_time) AS last_checkin
    FROM checkins AS c
    JOIN selected_business AS b
        ON b.business_id = c.business_id
),

```

```

hours_data AS (
    SELECT jsonb_object_agg(
        h.day_of_week,
        jsonb_build_object(
            'open_time', h.open_time,
            'close_time', h.close_time
        )
    ) AS data
FROM business_hours AS h
JOIN selected_business AS b
    ON b.business_id = h.business_id
),
attributes_data AS (
    SELECT jsonb_object_agg(ba.attribute_name,
ba.attribute_value) AS data
    FROM business_attributes AS ba
    JOIN selected_business AS b
        ON b.business_id = ba.business_id
)
SELECT
    b.*,
    (SELECT data FROM latest_reviews) AS latest_reviews,
    (SELECT data FROM latest_tips) AS latest_tips,
    (SELECT total_checkins FROM checkin_summary) AS
total_checkins,
    (SELECT first_checkin FROM checkin_summary) AS
first_checkin,
    (SELECT last_checkin FROM checkin_summary) AS last_checkin,
    (SELECT data FROM hours_data) AS hours,
    (SELECT data FROM attributes_data) AS attributes
FROM selected_business AS b;

```

MongoDB имплементација:

```

BID= db.businesses.find_one(sort=[("review_count", -1)], projection={"id":
1}) ["id"]

business_profile_card = bench("business_profile_card",
    lambda: db.businesses.find({"id": BID}))

```

Споредба на перформанси:

Метрика	PostgreSQL	MongoDB
Број на резултати	1	1
Минимално време (ms)	2714.371	2.23
Максимално време (ms)	2869.192	4.91
Медијана на времето на извршување (ms)	2821.35	2.83
Број на извршувања	10	10

За прашањето „Картичка со профил на деловен објект — еден објект + последни 50 рецензии со имиња на рецензентите + последни 10 препораки (tips) + вкупен број на пријавувања (check-ins) + работно време + карактеристики“, MongoDB покажува огромна предност во времето на извршување, при што **двете бази враќаат ист број на резултати (1)**. Медијаната кај PostgreSQL изнесува **2.821,35 ms**, додека кај MongoDB е само **2,83 ms**, што значи дека MongoDB ја извршува операцијата приближно **996 пати побрзо**. Слична разлика се забележува и кај минималното време (**2.714,371 ms** наспроти **2,23 ms**) и максималното време (**2.869,192 ms** наспроти **4,91 ms**). Бидејќи во овој случај и PostgreSQL и MongoDB враќаат **целосен и еднаков резултат**, предноста на MongoDB не е само во брзината, туку и во ефикасноста при составување на комплексен документ кој обединува податоци од повеќе поврзани структури, како рецензии, tips, check-ins, работно време и карактеристики.

3.2.4 Прашалник бр.7

Пронајди препораки напишани за ресторани во Филаделфија во текот на 2018 година кои добиле барем еден комплимент, вклучувајќи ги авторот, објектот, текстот на препораката и бројот на комплименти.

PostgreSQL имплементација:

```
SELECT
    t.tip_id, t.tip_date, t.text AS tip_text,
    t.compliment_count,
    u.user_id, u.name AS author_name,
    b.business_id, b.name AS business_name, b.address
FROM tips AS t
JOIN users AS u
    ON u.user_id = t.user_id
JOIN businesses AS b
    ON b.business_id = t.business_id
```

```

JOIN business_categories AS bc
  ON bc.business_id = b.business_id
WHERE b.city = 'Philadelphia'
  AND bc.category_name = 'Restaurants'
  AND t.compliment_count >= 1
  AND t.tip_date >= TIMESTAMP '2018-01-01'
  AND t.tip_date < TIMESTAMP '2019-01-01'
ORDER BY t.compliment_count DESC, t.tip_date DESC

```

MongoDB имплементација:

```

CITY = "Philadelphia"
YEAR_START = datetime(2018, 1, 1, tzinfo=timezone.utc) # inclusive
YEAR_END   = datetime(2019, 1, 1, tzinfo=timezone.utc) # exclusive

complimented_tips_philly_2018 = bench("complimented_tips_philly_2018",
  lambda: db.tips.aggregate([
    {"$match": {"date": {"$gte": YEAR_START, "$lt": YEAR_END},
      "compliment_count": {"$gte": 1}}},
    {"$lookup": {
      "from": "businesses",
      "localField": "business_id", "foreignField": "id",
      "as": "biz",
      "pipeline": [{"$match": {"location.city": CITY,
        "categories": "Restaurants"}},
        {"$project": {"_id": 0, "name": 1}}]}},
    {"$unwind": "$biz", # drops tips whose business did
not match
    {"$lookup": {
      "from": "users",
      "localField": "user_id", "foreignField": "id",
      "as": "author",
      "pipeline": [{"$project": {"_id": 0, "name": 1}}]}},
    {"$unwind": "$author"},
    {"$project": {"_id": 0,
      "author": "$author.name",
      "business": "$biz.name",
      "text": 1, "compliment_count": 1, "date": 1}},

```

```

        {"$sort": {"compliment_count": -1, "date": 1}},
    ], allowDiskUse=True),
)

```

Споредба на перформанси:

Метрика	PostgreSQL	MongoDB
Број на резултати	65	65
Минимално време (ms)	20.442	2130.07
Максимално време (ms)	25.59	2345.15
Медијана на времето на извршување (ms)	21.806	2175.9
Број на извршувања	10	10

За прашањето „Пронајди препораки напишани за ресторани во Филадельфија во текот на 2018 година кои добиле барем еден комплимент, вклучувајќи ги авторот, објектот, текстот на препораката и бројот на комплименти“, PostgreSQL покажува значително подобри перформанси од MongoDB, при што **двете бази враќаат ист број на резултати (65)**, што значи дека споредбата е направена врз целосно еквивалентен резултат. Медијаната на времето на извршување кај PostgreSQL изнесува само **21,806 ms**, додека кај MongoDB е **2.175,9 ms**, односно PostgreSQL ја извршува операцијата приближно **100 пати побрзо**. И минималното време кај PostgreSQL (**20,442 ms**) е значително пониско од MongoDB (**2.130,07 ms**), како и максималното време (**25,59 ms** наспроти **2.345,15 ms**). Овие резултати покажуваат дека за вакво барање, кое комбинира филтрирање според датум, тип на деловен објект и број на комплименти, како и поврзување со податоците за авторот и објектот, **PostgreSQL е значително поефикасен од MongoDB**.

3.3 Многу сложени прашалници

3.3.1 Прашалник бр.8

Елитни наспроти неелитни корисници: просечен број на фанови, пријатели, гласови и вкупен број на комплименти.

PostgreSQL имплементација:

```

WITH friend_counts AS (
    SELECT user_id, COUNT(*) AS friend_count
    FROM user_friends
    GROUP BY user_id
),

```

```

        compliment_counts AS (
            SELECT user_id, SUM(compliment_count) AS total_compliments
            FROM user_compliments
            GROUP BY user_id
        )
    SELECT
        CASE
            WHEN u.is_elite THEN 'Elite'
            ELSE 'Non-elite'
        END AS user_group,
        COUNT(*) AS user_count,
        ROUND(AVG(COALESCE(u.fans, 0)), 2) AS average_fans,
            ROUND(AVG(COALESCE(fc.friend_count, 0)), 2) AS
average_friends,
        ROUND(
            AVG(
                COALESCE(u.useful, 0)
                + COALESCE(u.funny, 0)
                + COALESCE(u.cool, 0)
            ),
            2
        ) AS average_total_votes,
        ROUND(AVG(COALESCE(cc.total_compliments, 0)), 2)
        AS average_total_compliments
    FROM users AS u
    LEFT JOIN friend_counts AS fc
        ON fc.user_id = u.user_id
    LEFT JOIN compliment_counts AS cc
        ON cc.user_id = u.user_id
    GROUP BY
        CASE
            WHEN u.is_elite THEN 'Elite'
            ELSE 'Non-elite'
        END
    ORDER BY user_group;

```

MongoDB имплементација:

```

elite_vs_rest_engagement = bench("elite_vs_rest_engagement",
    lambda: db.users.aggregate([

```

```

    {"$group": {
      "_id": "$elite.is_elite",
      "users": {"$sum": 1},
      "avg_fans": {"$avg": "$fans"},
      "avg_friends": {"$avg": "$friend_count"},
      "avg_reviews": {"$avg": "$review_count_total"},
      "avg_useful_votes": {"$avg": "$votes.useful"},
      "avg_compliments": {"$avg": {"$sum": [
        "$compliments.hot", "$compliments.more",
"$compliments.profile",
        "$compliments.cute", "$compliments.list",
"$compliments.note",
        "$compliments.plain", "$compliments.cool",
"$compliments.funny",
        "$compliments.writer", "$compliments.photos"]}}}},
    {"$project": {
      "_id": 0, "is_elite": "$_id", "users": 1,
      "avg_fans": {"$round": ["$avg_fans", 2]},
      "avg_friends": {"$round": ["$avg_friends", 2]},
      "avg_reviews": {"$round": ["$avg_reviews", 2]},
      "avg_useful_votes": {"$round": ["$avg_useful_votes", 2]},
      "avg_compliments": {"$round": ["$avg_compliments", 2]}}},
    {"$sort": {"is_elite": -1}},
  ], allowDiskUse=True)
)

```

Споредба на перформанси:

Метрика	PostgreSQL	MongoDB
Број на резултати	2	2
Минимално време (ms)	2660.566	16548.46
Максимално време (ms)	2929.634	18283.49
Медијана на времето на извршување (ms)	2747.31	17026.01
Број на извршувања	10	10

За прашањето „Елитни наспроти неелитни корисници: просечен број на фанови, пријатели, гласови и вкупен број на комплименти“, PostgreSQL покажува значително подобри перформанси од MongoDB, при што двете бази враќаат ист број на резултати.

Медијаната на времето на извршување кај PostgreSQL изнесува **2.747,31 ms**, додека кај MongoDB е **17.026,01 ms**, што значи дека PostgreSQL ја извршува операцијата приближно **6,2 пати побрзо**. И минималното време е значително пониско кај PostgreSQL (**2.660,566 ms** наспроти **16.548,46 ms**), како и максималното време (**2.929,634 ms** наспроти **18.283,49 ms**). Бидејќи двете бази враќаат ист број иста структура на резултати, овие мерења овозможуваат директна споредба и покажуваат дека **PostgreSQL е значително поефикасен за агрегирање и пресметување статистики над голем број кориснички записи**.

3.3.2 Прашалник бр.9

Пронајди го просечниот број на пријавувања (check-ins) по деловен објект за секоја категорија на дејност.

PostgreSQL имплементација:

```
WITH business_checkin_counts AS (  
    SELECT business_id, COUNT(*) AS checkin_count  
    FROM checkins  
    GROUP BY business_id  
)  
SELECT  
    bc.category_name,  
    ROUND(AVG(COALESCE(bcc.checkin_count, 0)), 2)  
    AS average_checkins_per_business  
FROM business_categories AS bc  
LEFT JOIN business_checkin_counts AS bcc  
    ON bcc.business_id = bc.business_id  
GROUP BY bc.category_name  
ORDER BY average_checkins_per_business DESC
```

MongoDB имплементација:

```
category_avg_checkins_per_business =  
bench("category_avg_checkins_per_business",  
    lambda: db.businesses.aggregate([  
        {"$unwind": "$categories"},  
        {"$group": {"_id": "$categories",  
            "businesses": {"$sum": 1},
```

```

        "avg_checkins": {"$avg": {"$ifNull":
["$checkin_stats.total", 0]}}}},
        {"$project": {"_id": 0, "category": "$_id", "businesses": 1,
        "avg_checkins": {"$round": ["$avg_checkins",
2]}}}},
        {"$sort": {"avg_checkins": -1}},
    ], allowDiskUse=True))

```

Споредба на перформанси:

Метрика	PostgreSQL	MongoDB
Број на резултати	1311	1311
Минимално време (ms)	1223.25	1812.01
Максимално време (ms)	1287.274	2410.8
Медијана на времето на извршување (ms)	1263.967	2050.98
Број на извршувања	10	10

За прашањето „Пронајди го просечниот број на пријавувања (check-ins) по деловен објект за секоја категорија на дејност“, PostgreSQL покажува подобри перформанси од MongoDB, при што **двете бази враќаат ист број на резултати (1.311)**, што овозможува директна споредба. Медијаната на времето на извршување кај PostgreSQL изнесува **1.263,967 ms**, додека кај MongoDB е **2.050,98 ms**, односно PostgreSQL ја извршува операцијата приближно **1,6 пати побрзо**. PostgreSQL има и пониско минимално време (**1.223,25 ms** наспроти **1.812,01 ms**) и максимално време (**1.287,274 ms** наспроти **2.410,8 ms**). Бидејќи бројот на резултати е идентичен кај двете бази, резултатите покажуваат дека **PostgreSQL е поефикасен за агрегирање и пресметување просечни вредности над поврзани податоци**, иако разликата во перформансите е помала во споредба со некои од претходните барања.

3.3.3 Прашалник бр.10

Пронајди ги 5-те најдобри деловни објекти во секој град врз основа на просечната оценка, земајќи ги предвид само објектите со најмалку 50 рецензии.

PostgreSQL имплементација:

```

WITH eligible_businesses AS (
    SELECT
        business_id, name, address, city, state,
        stars AS average_rating, review_count

```

```

        FROM businesses
        WHERE city IS NOT NULL
            AND stars IS NOT NULL
            AND review_count >= 50
    )
    SELECT
        b.business_id, b.name, b.address, b.city, b.state,
        b.average_rating, b.review_count
    FROM eligible_businesses AS b
    WHERE (
        SELECT COUNT(*)
        FROM eligible_businesses AS better_business
        WHERE better_business.city = b.city
        AND (
            better_business.average_rating > b.average_rating
            OR (
                better_business.average_rating = b.average_rating
                AND better_business.review_count > b.review_count
            )
            OR (
                better_business.average_rating = b.average_rating
                AND better_business.review_count = b.review_count
                AND better_business.business_id < b.business_id
            )
        )
    ) < 5
    ORDER BY
        b.city, b.average_rating DESC, b.review_count DESC,
b.business_id

```

MongoDB имплементација:

```

top5_per_city_min_50_reviews = bench("top5_per_city_min_50_reviews",
    lambda: db.businesses.aggregate([
        {"$match": {"review_count": {"$gte": 50}}},
        {"$sort": {"stars": -1, "review_count": -1}},
        {"$group": {"_id": "$location.city",
            "ranked": {"$push": {"business_id": "$id", "name":
"$name",
                                "stars": "$stars",

```

```

"review_count":
"$review_count"}}}},
  {"$project": {"_id": 0, "city": "$_id",
    "top_5": {"$slice": ["$ranked", 5]}},
  {"$sort": {"city": 1}},
], allowDiskUse=True))

```

Споредба на перформанси:

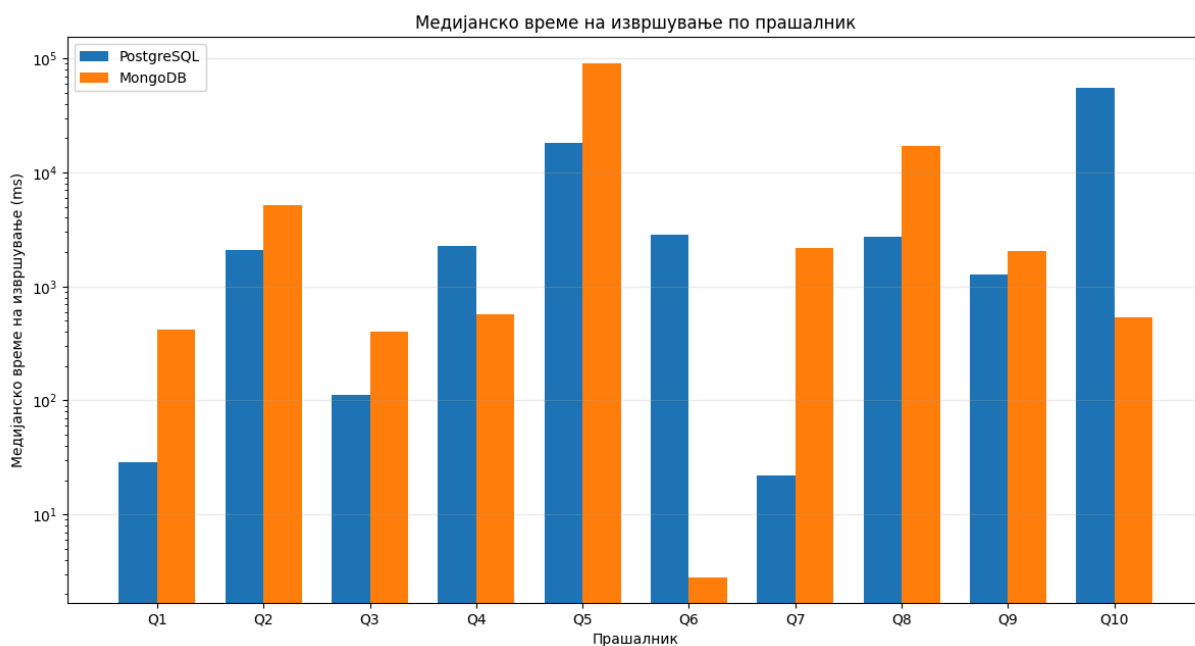
Метрика	PostgreSQL	MongoDB
Број на резултати	2179	449
Минимално време (ms)	51784.787	509.77
Максимално време (ms)	57800.882	593.55
Медијана на времето на извршување (ms)	55042.301	537.26
Број на извршувања	10	10

За прашањето „Пронајди ги 5-те најдобри деловни објекти во секој град врз основа на просечната оценка, земајќи ги предвид само објектите со најмалку 50 рецензии“, резултатите покажуваат интересна разлика помеѓу двете бази. MongoDB има значително пократко медијанско време на извршување од **537,26 ms**, во споредба со **55.042,301 ms** кај PostgreSQL, односно е приближно **102 пати побрз**. Сепак, PostgreSQL враќа **2.179 резултати**, додека MongoDB враќа само **449**, што е значително помал број. Бидејќи барањето е да се пронајдат **5-те најдобри објекти во секој град**, намалениот број резултати кај MongoDB укажува дека не се добиени сите релевантни резултати, односно дека постои компромис меѓу брзината и целосноста на резултатите. Поради тоа, иако MongoDB е драстично побрз во ова мерење, **PostgreSQL покажува предност во однос на целосноста на резултатите**, а времето на MongoDB не може само по себе да се земе како показател дека е подобар за ова барање.

3.4 Споредба што ги вклучува сите прашалници

Во целина, резултатите од десетте прашалници покажуваат дека **нема една база која е најбрза за сите типови операции**. PostgreSQL покажува подобри перформанси кај најголем дел од барањата за филтрирање, поврзување и агрегирање, додека MongoDB има значителна предност кај комплексните операции над документ-ориентирани податоци. Особено голема предност на PostgreSQL се забележува кај прашањата со поголем број резултати, како и кај операциите каде што се потребни сложени услови и поврзување на повеќе табели.

За појасна визуелна споредба, на Слика 3 е прикажано медијанското извршување за секој прашалник.

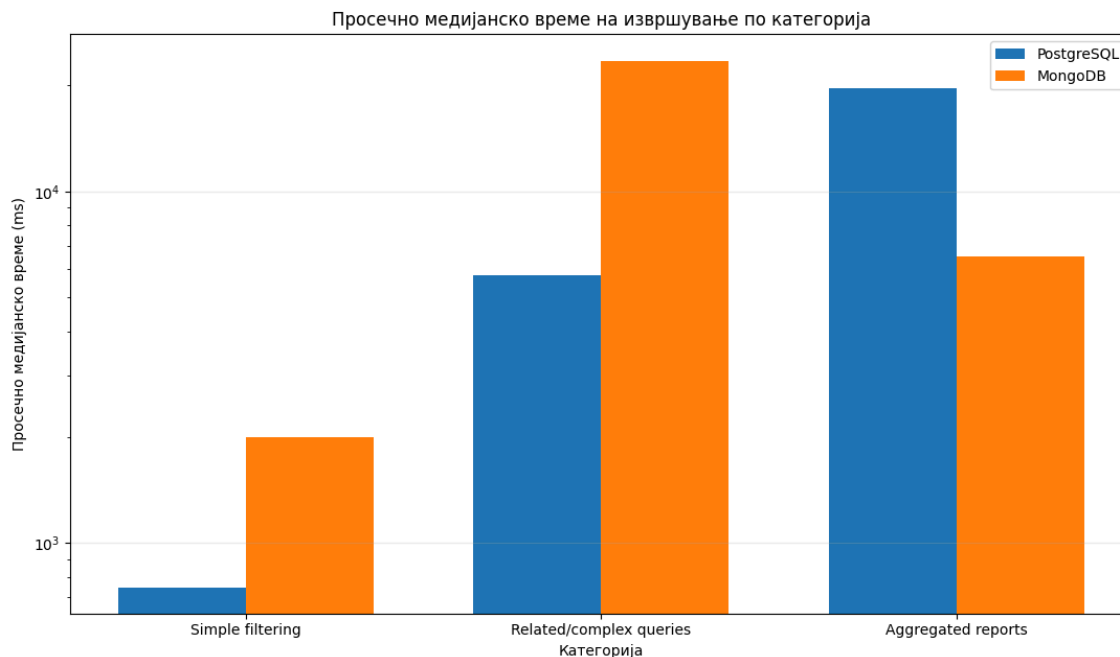


Слика бр.3 Визуелна компарација на временските перформанси на двете бази

Од друга страна, кај неколку комплексни прашалници MongoDB постигнува многу пократко време на извршување. Најизразен пример е **картичката со профил на деловен објект**, каде што медијаната е **2,83 ms** кај MongoDB, наспроти **2.821,35 ms** кај PostgreSQL, при што двете бази враќаат идентичен резултат. Сепак, кај други прашалници MongoDB има помал број резултати од PostgreSQL, на пример **25 наспроти 50**, **100 наспроти 690.265** и **449 наспроти 2.179**. Поради тоа, самото пократко време на извршување не може да се смета за предност доколку базата не ги враќа сите записи што ги исполнуваат поставените услови.

Генерално, резултатите покажуваат дека **PostgreSQL е посигурен избор кога приоритет се целосноста на резултатите и ефикасното обработување на големи релациски множества**, додека **MongoDB покажува голема предност кога податоците се организирани како комплексни документи и кога повеќе поврзани информации треба да се добијат како една целина**. Според тоа, изборот на база не треба да се темели само на времето на извршување, туку и на коректноста и целосноста на добиените резултати, како и на типот на операцијата што се извршува.

За подобро разбирање на перформансите според комплексноста на прашалниците, на Слика 4 е прикажана групирана анализа по категории.



Слика бр.4 Перформанси по категорија на прашалник

4. ДИСКУСИЈА

Резултатите покажуваат дека перформансите на PostgreSQL и MongoDB зависат од типот и сложеноста на прашалникот. PostgreSQL покажува подобри резултати кај најголем дел од прашалниците што вклучуваат филтрирање, поврзување на повеќе ентитети и агрегирање на голем број записи. Особено е ефикасен кога е потребно да се добијат целосни резултати од повеќе поврзани табели.

MongoDB покажува значителна предност кај прашалници каде податоците за еден бизнис се достапни како еден документ, како што е прикажувањето профил на деловен објект. Сепак, кај неколку прашалници MongoDB враќа помал број резултати од PostgreSQL, поради што пократкото време на извршување не може секогаш да се смета за директна предност. Затоа, при изборот на база треба да се земат предвид и брзината, и целосноста на резултатите.

5. ЗАКЛУЧОК

Во оваа семинарска работа беа споредени PostgreSQL и MongoDB врз Yelp податочното множество преку десет прашалници со различна сложеност. Резултатите покажуваат дека PostgreSQL е поефикасен кај најголем дел од прашалниците, особено при сложени операции со поврзување, групирање и агрегирање на податоци.

MongoDB покажува предност кај документ-ориентирани прашалници, каде повеќе информации се добиваат во рамки на еден документ. Оттука, PostgreSQL е посоодветен за аналитички операции над поврзани податоци, додека MongoDB е погоден за брзо читање на комплексно организирани документи.

6. КОРИСТЕНА ЛИТЕРАТУРА

1. PostgreSQL Official Documentation (Version 18). <https://www.postgresql.org/docs/18/>
2. MongoDB Official Documentation. <https://www.mongodb.com/docs/>
3. Yelp Dataset <https://www.kaggle.com/datasets/yelp-dataset/yelp-dataset>
4. A. Makris, K. Tserpes, G. Spiliopoulos, D. Zissis, and D. Anagnostopoulos, "MongoDB Vs PostgreSQL: A comparative study on performance aspects," *Geoinformatica*, vol. 25, pp. 243–268, 2021, doi: 10.1007/s10707-020-00407-w.
5. J. Urnikiene et al., "Comparative Read Performance Analysis of PostgreSQL and MongoDB in E-Commerce: An Empirical Study of Filtering and Analytical Queries," *Big Data and Cognitive Computing*, vol. 10, no. 2, Art. no. 66, 2026, doi: 10.3390/bdcc10020066.
6. Python Official Documentation <https://www.python.org/doc/>
7. Python Pandas Library. <https://pandas.pydata.org>